

Angular Moderno

14 - Lazy Loading com loadComponent



Angular Moderno

14 - Lazy Loading com loadComponent

Aprenda como carregar componentes apenas quando o usuário precisar deles, melhorando performance, escalabilidade e preparando o projeto para arquitetura profissional.

O Que Vamos Aprender Hoje



O que é Lazy Loading

Entender o conceito de carregamento sob demanda e por que ele é essencial em aplicações modernas.



Carregamento imediato vs. sob demanda

Comparar a abordagem tradicional com a estratégia moderna de divisão de bundle e ver o Impacto real no bundle da aplicação.



loadComponent no Angular 21+

Aplicar a API moderna do Router para implementar lazy loading por rota, sem NgModule.



Separação por feature e performance

Transformar cada feature em um chunk independente, preparando a base para projetos profissionais.

Onde Estamos no Projeto

Antes de evoluir, é importante entender o que já foi construído. O nosso e-commerce já possui uma estrutura sólida.

Features

Estrutura organizada por domínio: **home**, **produtos** e **carrinho** — cada um na sua pasta.

Angular Router

Rotas configuradas e funcionando com **provideRouter**. Navegação entre páginas sem reload.

Carrinho

Carrinho implementado com **Signals**. Estado ainda local. Isso é proposital e será refatorado em breve.

- ❗ O carrinho estar no componente de produtos é uma **etapa didática intencional**. Ele será extraído para um Service nas próximas aulas.



O Problema Atual

Como está hoje

Mesmo com rotas configuradas e componentes separados, todos os arquivos são **importados estáticamente no topo** do `app.routes.ts`. Isso significa que o navegador precisa baixar e processar o código de **todas as páginas** logo na primeira visita, independente de o usuário acessar aquela rota ou não.

Consequências

- Bundle inicial maior do que o necessário
- Tempo de carregamento mais lento
- Código de rotas nunca visitadas é executado
- Performance não escala com o crescimento da aplicação
- Má experiência em conexões lentas

⚠ Quanto mais features o projeto ganhar, **mais pesado** o bundle inicial se torna. Sem lazy loading, isso é inevitável, pois mesmo com rotas, ainda estamos carregando tudo de uma vez.



O Conceito de Lazy Loading

"Não carregamos tudo no início. Carregamos **quando o usuário precisa.**"

Lazy Loading (carregamento preguiçoso) é a estratégia de adiar o carregamento de recursos até o momento em que eles são realmente necessários. No contexto do Angular Router, isso significa que o código de um componente só é baixado do servidor quando o usuário navega para aquela rota específica. **Isso é possível graças ao `import()` dinâmico do JavaScript.**

Resultado imediato

Aplicação inicia mais rápida, pois o bundle inicial contém apenas o essencial.

Chunks separados

Cada feature vira um arquivo JS independente, carregado somente quando necessário.

Escalabilidade

Quanto mais features, mais você se beneficia. O bundle inicial não cresce proporcionalmente.

Antes: Carregamento Imediato

Veja como as rotas estão configuradas atualmente. O import estático no topo do arquivo é o que impede o lazy loading:

```
import { ListaProdutos } from './features/produtos/lista-produtos/lista-produtos';  
  
{ path: 'produtos', component: ListaProdutos }
```

Agora: Lazy Loading com `loadComponent`

A mudança é cirúrgica: substituímos `component:` por `loadComponent:` e removemos os imports estáticos do topo.

```
{  
  path: 'produtos',  
  loadComponent: () =>  
    import('./features/produtos/lista-produtos/lista-produtos').then(m => m.ListaProdutos)  
}
```

Passo 1 — Parar o Servidor

No terminal, pressione:

CTRL + C

Confirme com **S** ou **Y** se solicitado.

Por que parar antes de alterar?

O Angular CLI em modo `ng serve` observa alterações em tempo real. Modificar o `app.routes.ts` enquanto o servidor está rodando pode gerar erros de compilação intermitentes ou comportamentos inesperados durante a transição. **Parar o servidor antes é uma boa prática** que evita retrabalho.

- Evita erros de compilação parcial
- Garante rebuild limpo ao reiniciar
- Boa prática em mudanças estruturais

Passo 2 — Alterar o `app.routes.ts`

Vamos fazer uma **substituição completa** do arquivo de rotas. O objetivo é remover todos os imports estáticos do topo e converter cada rota para usar `loadComponent`.

- | | | |
|--|---|---|
| <div>1</div> <p>Abra o arquivo <code>app.routes.ts</code></p> <p>Localizado na raiz de <code>src/app/</code>. Este é o arquivo central de configuração de rotas.</p> | <div>2</div> <p>Apague todo o conteúdo atual</p> <p>Remova os imports estáticos e a configuração de rotas antiga por completo. Vamos escrever tudo do zero.</p> | <div>3</div> <p>Cole o código da próxima tela</p> <p>O novo código usa exclusivamente <code>loadComponent</code> com import dinâmico para todas as rotas.</p> |
|--|---|---|

⚠ Não esqueça de remover os imports do topo!
Se mantiver os imports estáticos, o lazy loading **não vai funcionar**.

Código Final do `app.routes.ts`

```
import { Routes } from '@angular/router';
export const routes: Routes = [
  {
    path: '',
    loadComponent: () => import('./features/home/home/home').then((m) => m.Home),
  },
  {
    path: 'produtos',
    loadComponent: () =>
      import('./features/produtos/lista-produtos/lista-produtos').then((m) => m.ListaProdutos),
  },
  {
    path: 'carrinho',
    loadComponent: () => import('./features/carrinho/carrinho/carrinho').then((m) => m.Carrinho),
  },
  {
    path: '**',
    redirectTo: '',
  },
];
```

O único import presente é `Routes` do `@angular/router`, que é um **tipo TypeScript**, não um componente. Isso é correto e necessário.

Explicação Técnica Aprofundada

`import()` é dinâmico (ES2020)

Diferente do `import` estático no topo do arquivo, o `import()` dinâmico é uma **função**. Ela só é executada quando chamada, não na inicialização do módulo. O bundler cria um arquivo JS separado para cada `import` dinâmico.

Retorna uma Promise

Como o carregamento é assíncrono (requer uma requisição HTTP), `import()` retorna uma `Promise`. O Angular Router sabe lidar com isso nativamente — ele aguarda a resolução antes de ativar a rota.

`.then(m => m.Componente)`

O argumento `m` representa o **módulo ES** exportado pelo arquivo. Como usamos `export class ListaProdutos` no componente, acessamos a classe diretamente via `m.ListaProdutos`.

Angular só carrega quando a rota é acessada

O Arrow Function passado para `loadComponent` só é invocado no momento da navegação. Na inicialização da app, essa função nem existe no bundle principal.

Remover Imports do Topo

Errado — import estático

```
// Isso ANULA o lazy loading!  
import { ListaProdutos }  
  from './features/produtos/  
    lista-produtos/  
    lista-produtos';  
  
export const routes: Routes = [  
  {  
    path: 'produtos',  
    loadComponent: () =>  
      import('...').then(...)  
  }  
];
```

Correto — sem imports de componentes

```
// Só importamos o tipo Routes  
import { Routes }  
  from '@angular/router';  
  
export const routes: Routes = [  
  {  
    path: 'produtos',  
    loadComponent: () =>  
      import('...').then(...)  
  }  
];
```

⊗ Se você mantiver o `import { ListaProdutos } from '...'` no topo, o bundler **vai incluir esse componente no bundle principal**, mesmo que você use `loadComponent`. O import estático sempre vence.

Passo 3 — Rodar o Projeto

Execute no terminal:

```
ng serve
```

Aguarde a compilação completar. Você verá a mensagem "**Application bundle generation complete**".

O que observar na compilação

Ao rodar `ng serve` após a alteração, preste atenção na saída do terminal. O Angular CLI vai listar os **chunks gerados** separadamente:

- `main.js` — bundle principal (menor agora)
- `chunk-HASH.js` — um arquivo para cada feature lazy
- Os tamanhos de cada arquivo são exibidos

Essa separação visual já é a **prova de que o lazy loading funcionou**.

Testes — Verificando o Funcionamento

Antes de usar o DevTools, confirme que a aplicação continua funcionando corretamente do ponto de vista do usuário:



Home funciona

Acesse `localhost:4200/` — a página inicial deve renderizar normalmente.



Produtos funciona

Navegue para `/produtos` — a lista de produtos deve aparecer sem erros no console.



Carrinho funciona

Acesse `/carrinho` — o componente deve carregar corretamente.



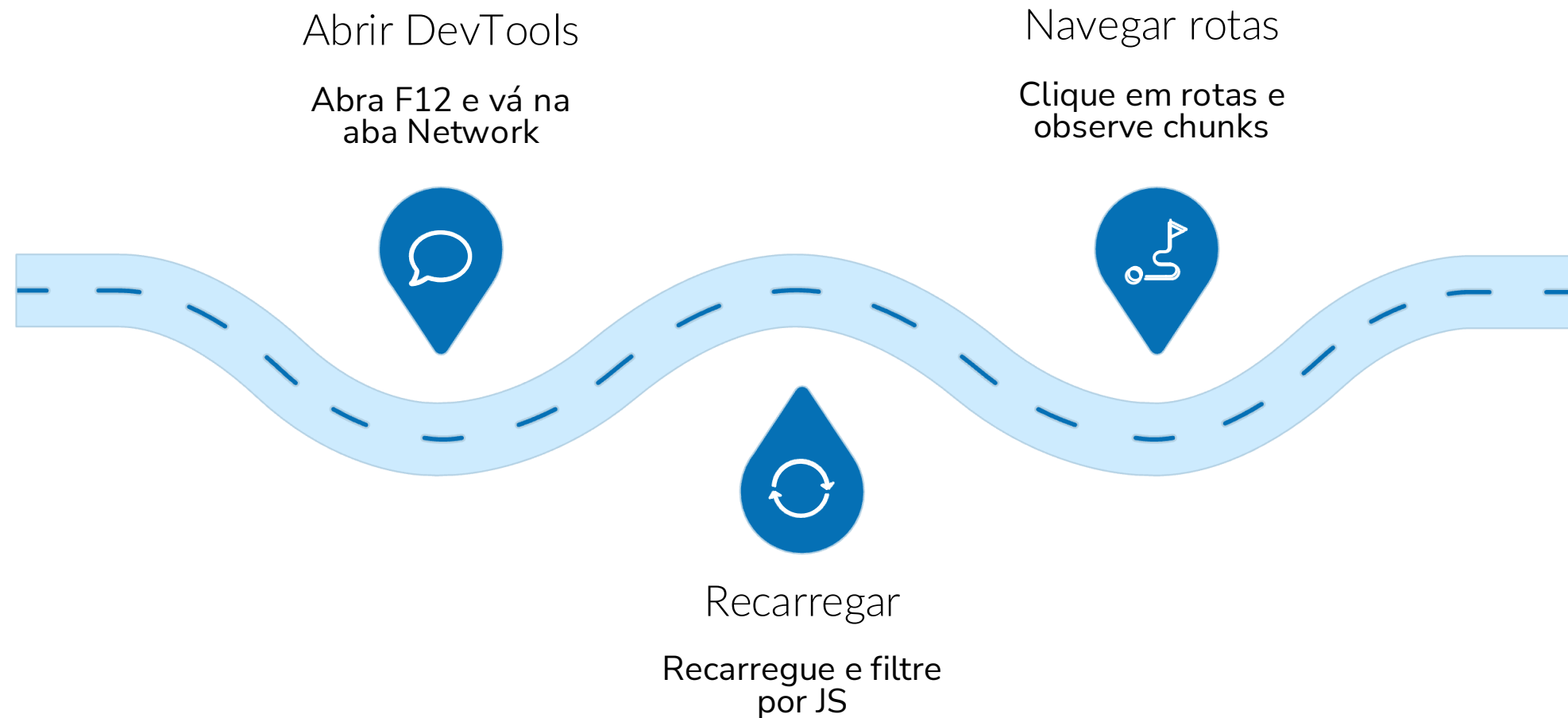
Navegação sem reload

Clique nos links de navegação — a troca de rota deve ser suave, sem recarregar a página.

✅ Se todas essas etapas funcionam, o lazy loading foi implementado com sucesso sem quebrar nenhuma funcionalidade existente.

Prova Real do Lazy Loading

Agora vamos **ver com os próprios olhos** o lazy loading acontecendo. Use o DevTools do navegador:



1. Abra o DevTools

Pressione **F12**, vá na aba **Network** e filtre por **JS**.

2. Recarregue a página

Veja quais arquivos JS são carregados na inicialização. Perceba que os chunks das features **não aparecem ainda**.

3. Clique em uma rota

Ao navegar para **/produtos**, um novo arquivo JS aparece na aba Network. Esse é o chunk lazy carregado sob demanda.

O Que Mudou na Prática

Antes — 1 Bundle Grande

main.js — em torno de 350 KB

Contém Home + Produtos + Carrinho + toda a lógica juntos. Tudo carregado na primeira visita.

Agora — Múltiplos Bundles

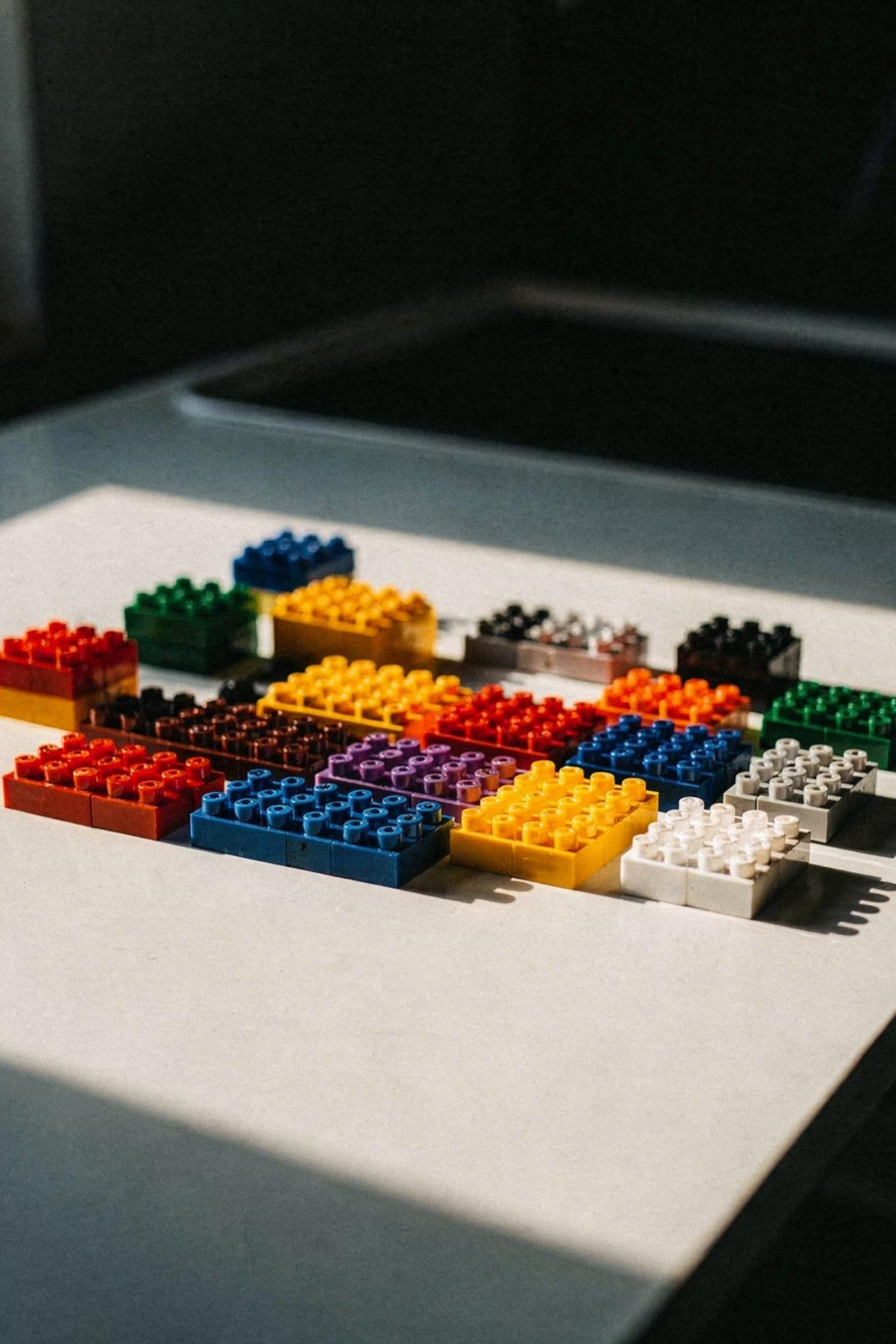
main.js — em torno de 80 KB (bundle principal mínimo)

chunk-produtos.js —
carregado ao acessar
`/produtos`

chunk-home.js —
carregado ao acessar `/`

chunk-carrinho.js —
carregado ao acessar
`/carrinho`

✓ Um usuário que visita apenas a home page baixa **apenas o chunk da home**, nunca o código de produtos ou carrinho.



Separação por Feature

O lazy loading não é apenas uma otimização de performance. Ele reforça e valoriza a **arquitetura por features** que adotamos desde o início do curso.

1

Pasta = Feature

Cada pasta em `features/` representa um domínio isolado da aplicação.

2

Feature = Chunk

Com lazy loading, cada feature gera um chunk JS separado no build final.

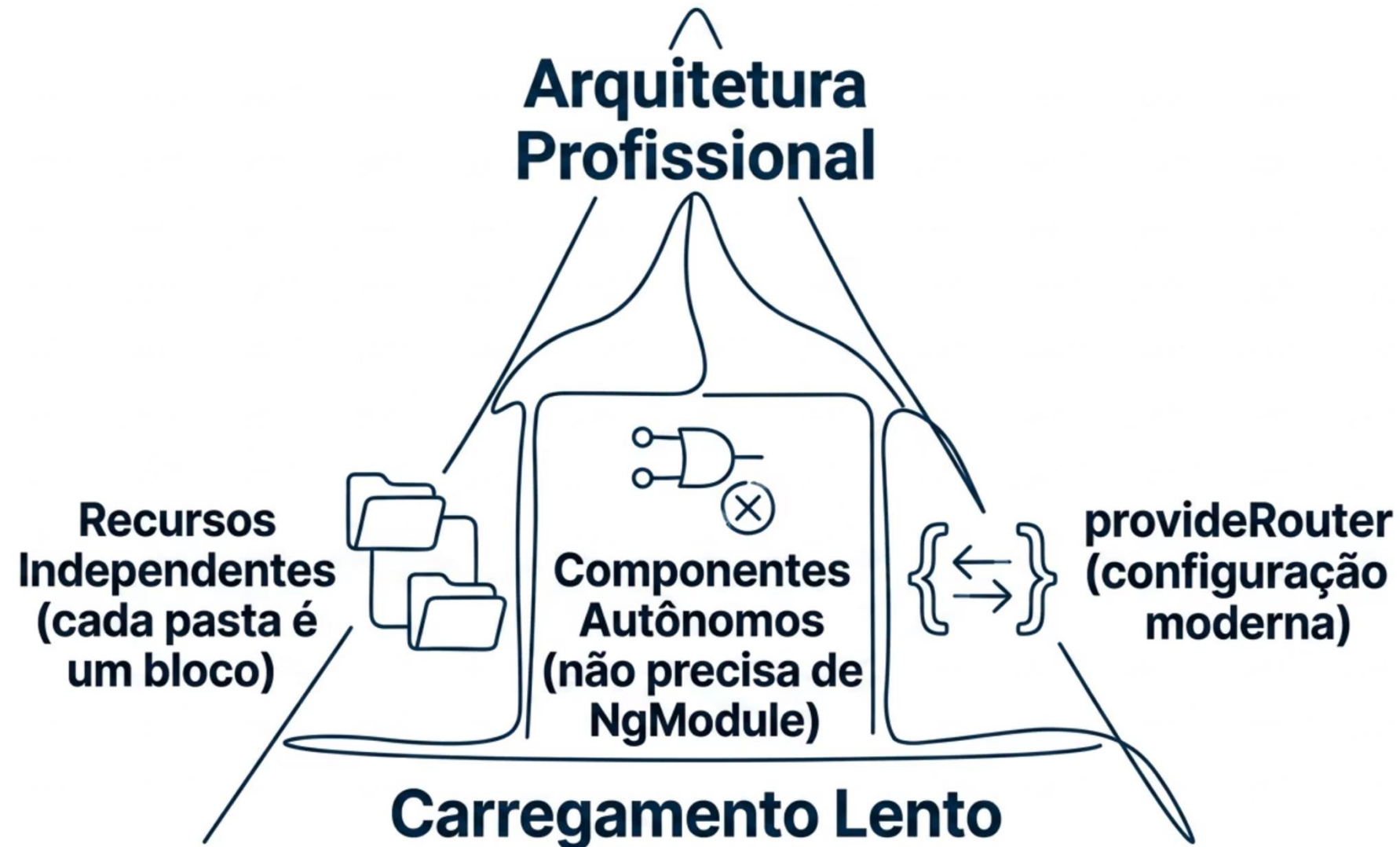
3

Chunk = Performance

Chunks menores significam carregamentos mais rápidos e experiência superior ao usuário.

Conexão com Arquitetura Profissional

A combinação de **Lazy Loading + Features independentes** é a base da arquitetura usada em projetos Angular de grande escala no mercado.



- ✔ Cada decisão que tomamos ao longo do curso (Standalone Components, estrutura por features, provideRouter) foi construindo os alicerces para este momento. Lazy loading é a cereja do bolo arquitetural.

Problema Intencional — O Carrinho

O que acontece agora

Com lazy loading, **cada rota cria uma nova instância** do componente quando acessada. Isso significa que o estado do carrinho — que ainda vive dentro do componente de produtos — é **perdido ao navegar para outra rota**.

Adicione produtos, navegue para a home, volte para produtos: o carrinho estará vazio novamente.

Por que isso é intencional?

- Esta é uma **etapa didática planejada**
- Demonstra o problema que Services resolvem
- Cria motivação real para o próximo conteúdo
- Reflete um problema real de projetos sem arquitetura

📌 Isso **não é um bug!**
É uma lição em andamento.

Conceito Crítico

"Componentes não compartilham estado entre rotas."

Cada vez que o Angular Router ativa uma rota com lazy loading, ele cria uma **nova instância** do componente. Quando você navega para fora, essa instância é destruída. Qualquer estado armazenado na propriedade do componente desaparece.

Estado no componente

Vive apenas enquanto o componente está ativo. Destruido ao navegar. **Não persiste entre rotas.**

Estado num Service

Vive enquanto a aplicação estiver aberta. Sobrevive a navegações. **Compartilhado entre componentes.**

Erros Comuns — Fique Atento

Estes são os erros mais frequentes ao implementar lazy loading pela primeira vez. Leia com atenção antes de testar:

1

Caminho errado no import

O caminho dentro do `import('...')` deve corresponder exatamente ao caminho do arquivo. Erros de case (maiúsculas/minúsculas) também causam falha. Copie o caminho do explorador de arquivos.

2

Nome errado do componente

Em `.then(m => m.NomeExato)`, o nome deve ser **idêntico** ao da classe exportada no arquivo do componente. Verifique a declaração `export class`.

3

Esquecer o `.then()`

Sem `.then(m => m.Componente)`, o Angular recebe o módulo ES inteiro, não a classe. A rota não vai funcionar e você verá erro de tipo no TypeScript.

4

Manter import estático no topo

Se o componente ainda estiver importado estaticamente no topo do arquivo, o lazy loading é anulado — o bundler o inclui no bundle principal de qualquer forma.

Regras do Curso — Reforço

Este curso segue padrões rígidos de código moderno. **Nunca use as abordagens abaixo**, independentemente de tutoriais antigos ou exemplos da internet:

Sem NgModule

Toda a configuração é feita via `bootstrapApplication` e `provideRouter`. NgModule é legado.

Sem RouterModule

Use apenas `provideRouter(routes)` no `app.config.ts`. `RouterModule.forRoot()` não é necessário.

Sem loadChildren

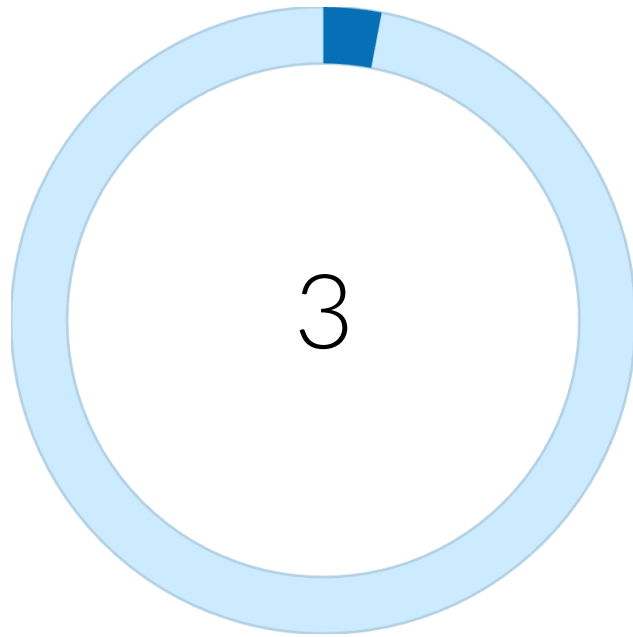
A API moderna usa `loadComponent` para Standalone Components. `loadChildren` era para NgModules.

Sem código legado

Use `@if` e `@for` em vez de `*ngIf` e `*ngFor`. A sintaxe de template moderna é obrigatória.

 Componentes já são **standalone por padrão** no Angular 21+. Não use `standalone: true` explicitamente, pois é redundante.

Conclusão da Aula

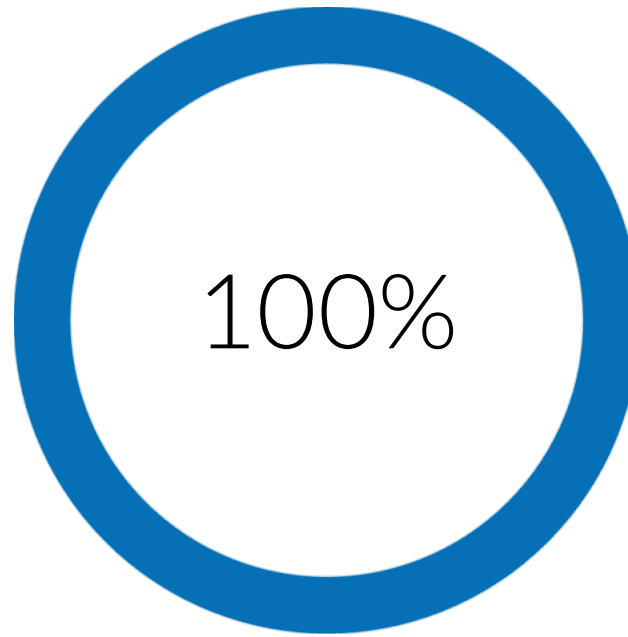


Features com lazy loading

Home, Produtos e Carrinho agora carregam sob demanda.

Lazy loading implementado

Todas as rotas usam `loadComponent` com import dinâmico.

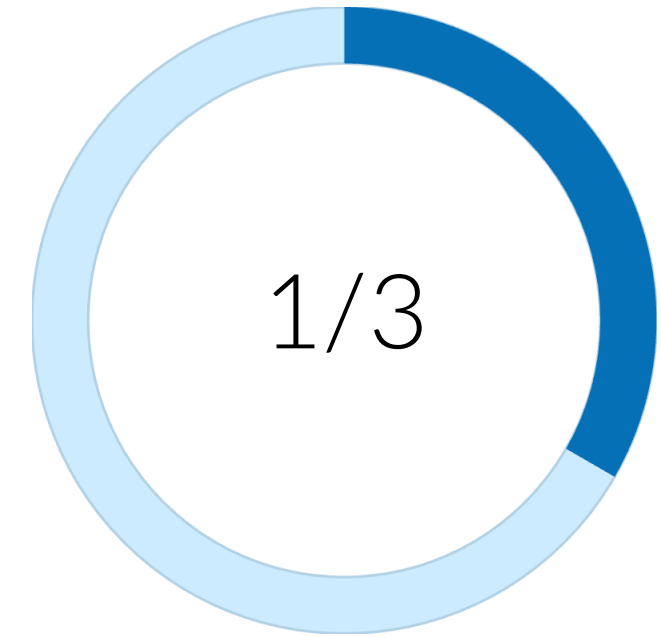


Standalone

Nenhum NgModule utilizado. Arquitetura completamente moderna.

Performance melhorada

Bundle inicial menor, chunks independentes por feature.



Bundle inicial

Estimativa: apenas a fração do bundle anterior é carregada no início.

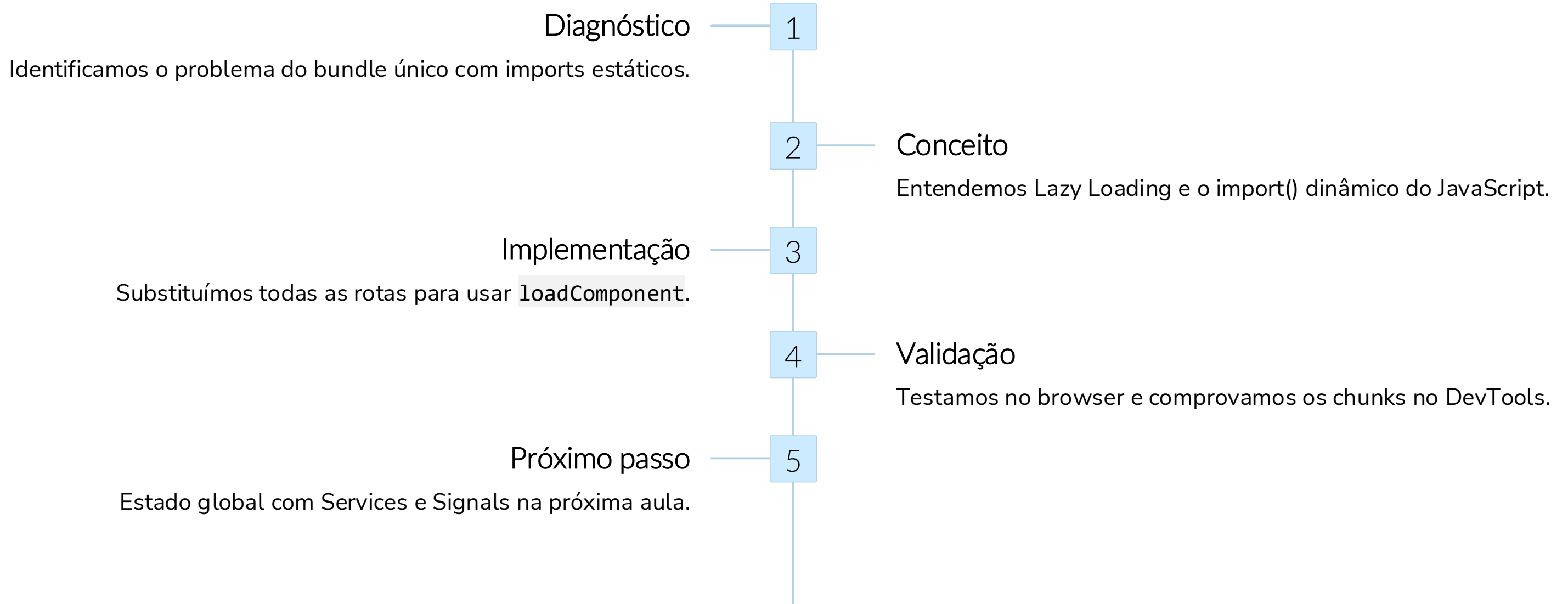
Arquitetura evoluída

Projeto preparado para crescer sem impacto no carregamento inicial.



"Agora cada
página é
carregada apenas
quando o usuário
precisa dela."

Recapitulação — O Que Fizemos



✓ O projeto de e-commerce evoluiu mais um passo rumo à **arquitetura profissional Angular Moderno**. Sem alterar nenhum componente, apenas o Router!

Conceitos-chave desta aula:

- `loadComponent` — API de lazy loading por rota
- `import()` — import dinâmico ES2020
- `.then(m => m.Classe)` — extração do componente
- Chunk — arquivo JS gerado por feature lazy
- Bundle principal — código mínimo de inicialização
- Estado local vs. estado global — motivação para Services

Dúvidas

?

